

The $N \times M$ Integration Problem, Worked Out by Hand

A complete numerical walk-through of how a shared protocol collapses $M \times N$ bespoke integrations down to $M + N$ — every count shown.

What this document does. It takes one concrete AI ecosystem — $M = 5$ clients that each want to reach $N = 20$ backends — and counts *every* integration you must build and maintain, first the old bespoke way and then the MCP way. Nothing is hand-waved. Each number below is produced by a tiny Python reference implementation (Appendix A), so the arithmetic is exact and self-consistent. The headline fact of Module 0.0 — “the count collapses from $M \times N$ to $M + N$ ” — is derived here as an exact algebraic identity, not an analogy.

How to read it. Every component of the argument is explained on its own and gets a *Mini-example* box isolating that idea on the smallest possible numbers, alongside the running 5×20 example. *Intuition* boxes say *why* the math works; *Watch out* boxes flag where the clean count meets messy reality.

Concepts covered, in order: pairwise integration cost · the integration matrix · complete bipartite graphs $K_{M,N}$ · the hub-and-spoke (star) collapse · the savings identity $(M - 1)(N - 1)$ · break-even · marginal cost and the linear/quadratic gap · symmetric growth n^2 vs. $2n$ · the harmonic-mean speed-up.

Internal learning material. Numbers are reproducible from the appendix code. v1.0

1 The setup: an ecosystem small enough to count by hand

Real organisations run dozens of AI surfaces against hundreds of backends; you cannot count those edges on paper. So we fix the exact ecosystem that Module 0.0 names and count it in full. The mechanics are identical at scale — only the two numbers grow.

Quantity	Symbol	Value here
AI clients (host-side connectors)	M	5
Backends to reach (servers)	N	20
A single client–server connector	—	one “integration”
Cost model	—	one unit per integration built & maintained

The clients ($M = 5$): Claude Desktop, Cursor, an in-house agent, a Slack copilot, an automation worker. **The backends** ($N = 20$): GitHub, Jira, Postgres, S3, internal wiki, payments API, Slack, Gmail, Google Drive, Notion, Linear, Stripe, Salesforce, Snowflake, Elasticsearch, Redis, Kubernetes, Datadog, PagerDuty, Confluence.

Intuition

A language model is a pure function `tokens_in` → `tokens_out`: it owns no socket, no file descriptor, no clock. Anything it *does* in the world is done by something else — a connector between an AI client and a backend. So the real engineering cost of “giving models tools” is the cost of *building and maintaining those connectors*. This document counts exactly that cost, and shows what a shared protocol does to it.

1.1 Notation

We count *integrations* — distinct pieces of connector code that must be written, authenticated, error-handled, and kept alive as upstream APIs drift. We write B for the bespoke (no-protocol) count and P for the protocol count. Every displayed number is an exact integer.

2 Step 1 — The bespoke world: the integration matrix

Without a shared protocol, each client must speak each backend’s private dialect: its own auth, its own tool-schema convention, its own error shape. A connector written for one client *cannot* be lifted into another without a rewrite. So every (client, backend) pair is its own project. Lay them out as a matrix — rows are clients, columns are backends, and *every cell is a separate integration you own*:

	GitHub	Jira	Postgres	S3	...	row total
Claude Desktop	✓	✓	✓	✓	...	20
Cursor	✓	✓	✓	✓	...	20
In-house agent	✓	✓	✓	✓	...	20
Slack copilot	✓	✓	✓	✓	...	20
Automation worker	✓	✓	✓	✓	...	20
total						100

The matrix has M rows and N columns, every cell filled. The bespoke count is therefore the area of the rectangle:

$$B = M \times N = 5 \times 20 = \boxed{100}.$$

Mini-example — this operation in isolation

Take the smallest matrix that is still a rectangle: $M = 2$ clients, $N = 3$ backends. The grid is

$$\begin{array}{c|ccc} & s_1 & s_2 & s_3 \\ \hline c_1 & \bullet & \bullet & \bullet \\ c_2 & \bullet & \bullet & \bullet \end{array} \implies B = 2 \times 3 = 6.$$

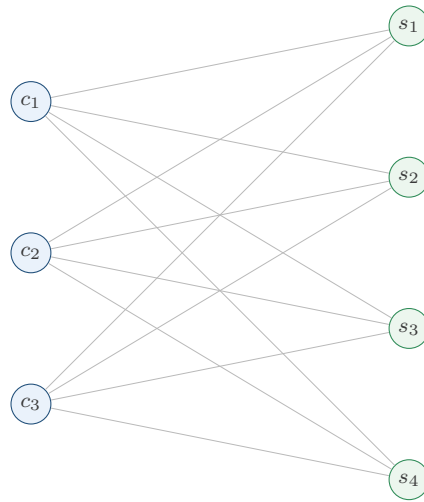
Six dots, six connectors. Add a column (a 4th backend) and you add a *whole column* of dots — one per existing client. That “add a whole row/column” behaviour is the entire problem, in miniature.

Intuition

Why multiplication, not addition? Because the integrations do not *compose*. Knowing how Cursor talks to GitHub teaches you nothing reusable about how the Slack copilot talks to GitHub — different host, different plumbing. With no shared interface, effort is spent *per pair*, and the number of pairs in an M -by- N grid is $M \cdot N$.

3 Step 2 — The same cost as a graph: $K_{M,N}$

Model each client and each backend as a node, and draw an edge for every integration. The bespoke world connects *every* client to *every* backend, which is exactly the **complete bipartite graph** $K_{M,N}$. A standard fact: $K_{M,N}$ has $M \cdot N$ edges — one per (left, right) pair. So “number of integrations” and “number of edges in $K_{M,N}$ ” are the same number, 100.



Bespoke: $K_{3,4}$
 $3 \times 4 = 12$ edges

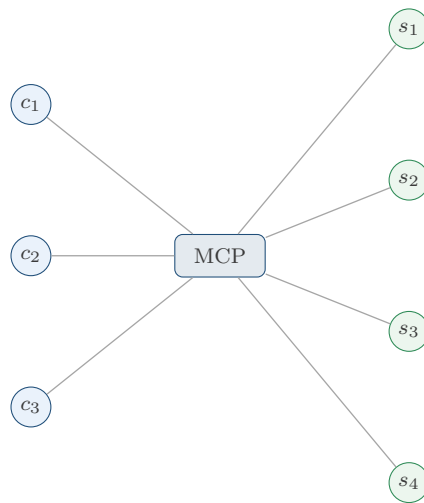
(Drawn at $3 \times 4 = 12$ edges so the lines stay readable; the running example is $5 \times 20 = 100$.)

Watch out

The graph also exposes the *maintenance* cost the headline number hides. When one backend changes its API, *every* edge touching that node rots at once — that is M connectors to patch, not one. The matrix is not just expensive to build; it is expensive to keep alive, and the blast radius of any single upstream change is a whole row or column.

4 Step 3 — The MCP move: collapse the graph to a star

Introduce one shared interface — a wire protocol. Now each client speaks the protocol *once* (it can then reach anything that speaks it), and each backend is wrapped in an MCP server *once* (it is then reachable by anything that speaks it). Structurally, we have inserted a hub: every node connects to the *protocol*, not to every node on the far side. The complete bipartite graph becomes a **star** (hub-and-spoke).



Protocol: star
 $3 + 4 = 7$ spokes

Each of the M clients contributes one spoke; each of the N backends contributes one spoke. The protocol count is therefore the *perimeter*, not the area:

$$P = M + N = 5 + 20 = \boxed{25}.$$

The MCP move

“USB-C for AI.” One client implements the wire protocol once; one server wraps a backend once; capability negotiation at connect-time lets clients discover tools dynamically, so nobody is recompiled to learn a new tool. The 12-edge mesh ($K_{3,4}$) becomes a 7-spoke star — and at the real scale, the 100-edge mesh becomes a 25-spoke star.

Intuition

The hub turns “everyone learns everyone’s dialect” ($M \cdot N$ private conversations) into “everyone learns one shared language” ($M + N$ translations into that language). You pay once per participant to join the protocol, instead of once per *pair* of participants. Area $\rightarrow$ perimeter is the whole game.

5 Step 4 — The savings, as an exact identity

The reduction is the difference between area and perimeter:

$$\text{savings} = B - P = MN - (M + N).$$

This factors exactly. Add and subtract 1:

$$MN - M - N = \underbrace{MN - M - N + 1}_{(M-1)(N-1)} - 1 = (M-1)(N-1) - 1.$$

So the saved integrations are

$$\boxed{B - P = (M-1)(N-1) - 1}.$$

Check it on the running example: $(5-1)(20-1) - 1 = 4 \cdot 19 - 1 = 76 - 1 = 75$. And indeed $100 - 25 = 75$. ✓

That is a $75/100 = 75\%$ reduction, or a factor of $100/25 = 4.0\times$ fewer integrations to own.

Mini-example — when is the protocol worth it?

The protocol only helps when savings > 0 , i.e. $(M-1)(N-1) > 1$. Walk the smallest cases:

$M \times N$	$B = MN$	$P = M+N$	$B - P$
2×2	4	4	0 (a wash)
2×3	6	5	+1
3×3	9	6	+3

Break-even is exactly $(M-1)(N-1) = 1$, i.e. $M = N = 2$. From the very next step (2×3) onward the protocol always wins, and the lead widens fast.

Watch out

The clean identity assumes the protocol layer is “free” once standardised — that wrapping a backend in an MCP server is the same unit of work as a bespoke connector. In reality

there is a one-time cost to adopt the protocol (SDKs, auth, the spec itself). The lesson is not “ $M + N$ is literally zero-overhead”; it is that $M + N$ *grows* like the perimeter while MN grows like the area — so for any non-trivial ecosystem the gap dominates the fixed cost.

6 Step 5 — Marginal cost: the real win

The headline ratio understates the benefit. What teams actually feel is the cost of *adding one more* surface to an existing ecosystem. Differentiate both counts.

Bespoke: $B(M, N) = MN$, so adding one client costs $\partial B / \partial M = N$ new integrations, and adding one backend costs $\partial B / \partial N = M$. Protocol: $P(M, N) = M + N$, so adding either costs exactly 1.

Action on the 5×20 ecosystem	Bespoke adds	Protocol adds
Add a 6 th client (reach all 20 backends)	$N = 20$	1
Add a 21 st backend (reach all 5 clients)	$M = 5$	1

Intuition

This is the difference that compounds. In the bespoke world a new client is a 20-connector project before it is useful; a new backend is a 5-connector project. With a protocol, a new client writes *one* thing and instantly reaches all 20 existing backends; a new backend writes *one* thing and is instantly reachable by all 5 existing clients. Marginal cost falls from “proportional to the other side” to constant. That constant marginal cost is why ecosystems — not just individual integrations — form around a protocol.

7 Step 6 — Asymptotics: quadratic vs. linear

Let the ecosystem grow symmetrically, $M = N = n$. Then

$$B = n^2, \quad P = 2n, \quad \frac{B}{P} = \frac{n^2}{2n} = \frac{n}{2}.$$

The bespoke cost is $\Theta(n^2)$; the protocol cost is $\Theta(n)$. The speed-up factor $n/2$ has no ceiling — the larger the ecosystem, the more lopsided the comparison.

n	$B = n^2$	$P = 2n$	savings $B - P$	ratio $n/2$
1	1	2	−1	0.5
2	4	4	0	1.0
3	9	6	3	1.5
4	16	8	8	2.0
5	25	10	15	2.5
6	36	12	24	3.0
8	64	16	48	4.0
10	100	20	80	5.0

Mini-example — the harmonic-mean speed-up

For the asymmetric case the speed-up has a tidy closed form. The factor is

$$\frac{B}{P} = \frac{MN}{M+N} = \frac{1}{\frac{1}{M} + \frac{1}{N}} = \frac{1}{2}H(M, N),$$

where $H = \frac{2MN}{M+N}$ is the harmonic mean of M and N . For $M = 5, N = 20$: $H = 2 \cdot 100/25 = 8$, so the speed-up is $H/2 = 4.0\times$ — matching $100/25$ exactly. The harmonic mean leans toward the *smaller* of the two, so a lopsided ecosystem (few clients, many backends) is gated by the small side.

7.1 Asymmetric growth (the common real shape)

Most ecosystems have far more backends than clients. Fix $N = 20$ and let the number of clients M grow:

M	$B = 20M$	$P = M + 20$	savings	reduction = $\frac{B-P}{B}$
1	20	21	−1	−5.0%
2	40	22	18	45.0%
3	60	23	37	61.7%
4	80	24	56	70.0%
5	100	25	75	75.0%
6	120	26	94	78.3%
7	140	27	113	80.7%
8	160	28	132	82.5%

At $M = 1$ the protocol is a slight *loss* (21 vs. 20): with a single client there are no shared edges to amortise. By $M = 2$ it is already a 45% reduction, and the reduction climbs toward 100% as M grows.

8 An honest word about these numbers

The counting model treats every integration as one identical unit. Reality is lumpier: a Postgres connector is not the same effort as a Slack connector, the protocol carries real adoption overhead, and an organisation rarely needs the *full* matrix (not every client must reach every backend). None of that changes the shape of the result.

Watch out

A worked count with unit-cost edges shows you the *structure* — area versus perimeter, $\Theta(n^2)$ versus $\Theta(n)$, constant marginal cost. It cannot tell you the dollar cost of any one connector. The right mental model: *the protocol does not make any single integration cheaper; it changes how the total scales as the ecosystem grows*. That is why MCP is framed as infrastructure, not as a connector library — and why “USB-C for AI,” though marketing, is structurally accurate.

The practical consequence Module 0.0 draws: once integration cost is linear, the interesting work moves *up* the stack — to which tools should exist, who may call them, and what happens when they fail. That is the rest of this course.

9 The argument, at a glance

#	Idea	Formula	Value (5×20)
1	Bespoke pairwise count (matrix area)	$B = MN$	100
2	Same cost as edges of $K_{M,N}$	$ E = MN$	100
3	Protocol collapses mesh to a star	$P = M + N$	25
4	Savings identity	$B - P = (M - 1)(N - 1) - 1$	75
4	Reduction fraction	$(B - P)/B$	75%
5	Marginal cost of one new client	$\partial B/\partial M = N$ vs. 1	20 vs. 1
6	Asymptotic cost (symmetric n)	n^2 vs. $2n$	—
6	Speed-up factor	$MN/(M+N) = \frac{1}{2}H$	$4.0\times$

10 Equation cheat-sheet

Bespoke integration count	$B = M \cdot N$
Edges of complete bipartite graph	$ E(K_{M,N}) = M \cdot N$
Protocol integration count (star)	$P = M + N$
Savings (exact identity)	$B - P = (M - 1)(N - 1) - 1$
Break-even	$(M - 1)(N - 1) = 1 \iff M = N = 2$
Reduction fraction	$\frac{B - P}{B} = 1 - \frac{M + N}{MN}$
Marginal cost, bespoke	$\frac{\partial B}{\partial M} = N, \quad \frac{\partial B}{\partial N} = M$
Marginal cost, protocol	$\frac{\partial P}{\partial M} = \frac{\partial P}{\partial N} = 1$
Symmetric growth ($M = N = n$)	$B = n^2, P = 2n, B/P = n/2$
Speed-up factor	$\frac{B}{P} = \frac{MN}{M + N} = \frac{1}{2}H(M, N)$

A Reference implementation

Every number in this document is produced by the following self-contained Python program. Run it to reproduce all counts; change **M** and **N** to explore any ecosystem.

```

def integration_counts(M, N):
    B = M * N                                # bespoke: every (client, server) pair
    P = M + N                                # protocol: one spoke per participant
    savings = B - P
    assert savings == (M - 1) * (N - 1) - 1    # the identity
    return dict(
        bespoke=B, protocol=P, savings=savings,
        reduction=savings / B if B else 0.0,
        speedup=B / P,                        # == 0.5 * harmonic_mean(M, N)
        marginal_client_bespoke=N, marginal_client_protocol=1,
        marginal_server_bespoke=M, marginal_server_protocol=1,
    )

print(integration_counts(5, 20))
# {'bespoke': 100, 'protocol': 25, 'savings': 75,
#   'reduction': 0.75, 'speedup': 4.0,
#   'marginal_client_bespoke': 20, 'marginal_client_protocol': 1,
#   'marginal_server_bespoke': 5, 'marginal_server_protocol': 1}

# growth table (symmetric n)
for n in range(1, 11):
    print(n, n*n, 2*n, n*n - 2*n)

```

— end of document —